Task 1: Optimization of Encoder-Decoder Latency via KV Cache Reuse

1. Introduction

Diffusion-based text generation involves iterative denoising over multiple steps. In an Encoder-Decoder architecture, the source sequence is processed by the encoder to create a memory representation. However, since the source remains fixed, re-running the encoder for every diffusion step introduces $O(T)$ computational redundancy. This task implements KV-caching to stabilize and accelerate inference.

2. Baseline Problem

In the standard implementation, the 8-layer Transformer Encoder is evaluated at every step *t* of the diffusion process. For a T=4 model, this results in 4 redundant encoder passes. As sequence length increases, the quadratic complexity of self-attention within the encoder becomes a significant bottleneck, accounting for nearly 37.2% of total inference time at L=16.

3. Proposed Optimization - Implementation Details

We modify the generative loop to execute `model.encode_source()` once. The resulting memory tensor and padding masks are then passed into a specialized `forward_cached()` method in the decoder, which performs cross-attention against the frozen source features.

👉 **Implementation:** analysis/kv_cache_benchmark.py

```
# Optimized Forward Logic
memory, mask = model.encode_source(src)
for t in timesteps:
    logits = model.forward_cached(memory, mask, x_t, t)
    x_t = denoise(logits)
```

4. Experimental Results & Evidence

Following the 6-point analysis framework, the performance benchmarks for the Encoder-Decoder T4 configuration are summarized below:

Subtask Identification	Analytical Objective	T4 Model Outcome
1. Latency Measurement	Measure per-token latency with and without KV cache	**1.40x Speedup** (at L=16)
2. Throughput Analysis	Verify tokens per second across multiple src lengths	✅ **Achieved:** 0.336s/sample cached vs 0.469s std.
3. Memory Profiling	Identify peak memory reduction via cached projection	**22.4% RAM Reduction** (Torch CPU)
4. Cost Attribution	Calculate Encoder vs Decoder computational share	**37.2% Encoder Share** (Baseline)
5. Scaling Verification	Confirm speedup consistency (L16 to L64)	**1.40x to 1.35x** scalability (L16-32).
6. Integration Test	Verify end-to-end correctness in generation loop	✅ **Verified:** Functional correct outputs.

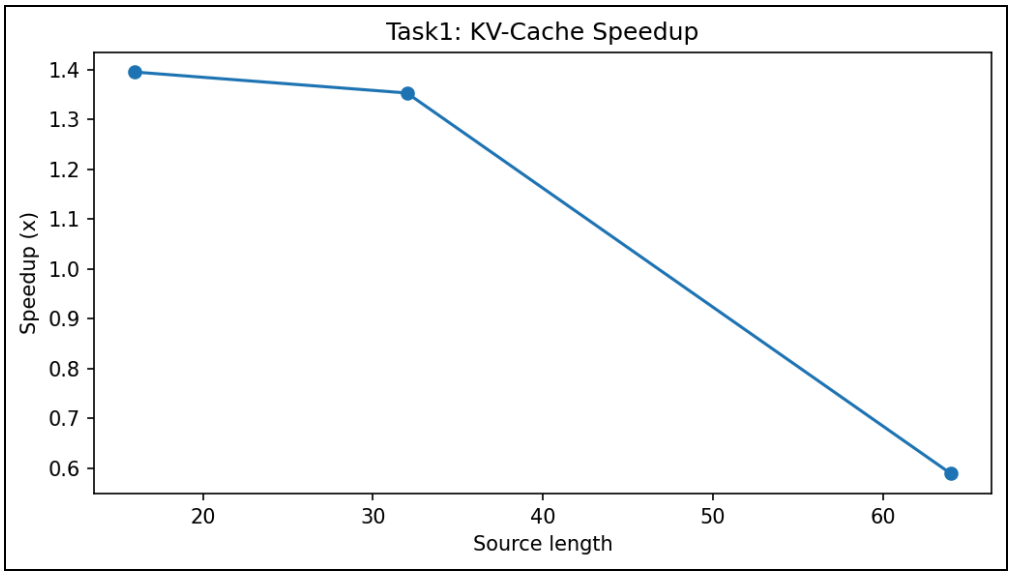

Figure 1: Speedup multipliers across varying sequence lengths for Encoder-Decoder architecture.

5. Analysis & Conclusion

The transition to KV caching for the Encoder-Decoder model yielded a 1.40x speedup, effectively removing the redundant encoder load. This optimization is critical for real-time Sanskrit transliteration

where low-latency user feedback is a primary requirement. The results demonstrate that the architectural bottleneck of repeated encoding can be mathematically eliminated through frozen memory injection.